VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

$\sim \cdots * \cdots \sim$



Computer Engineering Project

Week 1 Progress Report

REAL-TIME PERSONAL PROTECTIVE EQUIPMENT (PPE) MONITORING SYSTEM USING EDGE AI AND IOTs

Instructor: **Dr. Vo Tuan Binh**

Name	ID
Pham Ngoc Mai Anh	2352063
Tran Quoc Hao	2352299
Nguyen Minh Khue	2352631

Ho Chi Minh City, 2026

Contents

1 Introduction and Problem Formulation **3**
1.1 Industrial Context and Problem Statement . . . 3
1.2 Problem Statement . . . 3
1.3 Proposed System Architecture and Objectives . . . 4

2 Theoretical Foundations **5**
2.1 Theoretical Basis of Object Detection Architectures . . . 5
2.1.1 Two-Stage vs. One-Stage Detectors . . . 5
2.1.2 Detection Loss and Optimization Metrics . . . 5
2.2 Theory of Edge Hardware and Model Quantization . . . 5
2.2.1 CPU vs. GPU Embedded Architectures . . . 6
2.2.2 Post-Training Quantization (PTQ) Theory . . . 6
2.3 Internet of Things (IoT) Telemetry: The MQTT Protocol . . . 6
2.4 Theoretical Requirements for Computer Vision Datasets . . . 7

3 Theoretical Foundations of Deep Learning and System Architecture **8**
3.1 Evolution of Object Detection Paradigms . . . 8
3.2 Core Components of Modern Object Detection . . . 8
3.3 Anatomy of a Neural Network Architecture . . . 9
3.4 Architectural Evolution of YOLO Models . . . 10
3.4.1 YOLOv1 (2016): The "You Only Look Once" Paradigm . . . 10
3.4.2 YOLOv2 (2017): The Anchor Box Revolution . . . 11
3.4.3 YOLOv8 (2023): Modular Architecture and the Return to Anchor-Free . . . 12
3.4.4 YOLO26 (2026): Unified End-to-End Vision Models . . . 13
3.5 Comprehensive Architectural Comparison . . . 14

4 Edge Platforms, Model Optimization **16**
4.1 Edge computing . . . 16
4.2 Edge devices . . . 17
4.3 Edge AI deployment . . . 18

4.4 Model optimization 19
4.4.1 Quantization 19
4.4.2 Pruning 20

1 Introduction and Problem Formulation

1.1 Industrial Context and Problem Statement

In modern industrial automation and civil construction, occupational health and safety (OHS) compliance is a critical priority. Globally, the construction and heavy manufacturing sectors remain highly hazardous. According to the International Labor Organization (ILO), at least 60,000 fatal accidents occur on construction sites worldwide each year, equating to one fatal accident every ten minutes [1].

It is crucial to note that the lack of Personal Protective Equipment (PPE) is not the root cause of these occupational accidents. Workplace incidents are primarily triggered by structural failures, heavy machinery malfunctions, or falls. However, PPE—specifically hard hats and high-visibility vests—serves as the critical last line of physical defense. When workers are unequipped, the severity of these incidents is drastically amplified, frequently transforming otherwise survivable accidents into fatal head traumas or deadly machinery collisions.

In Vietnam, occupational safety is strictly governed by the **Law on Occupational Safety and Health (Law No. 84/2015/QH13)** [2]. This legal framework explicitly mandates employers to provide adequate protective equipment and obligates workers to strictly utilize it in hazardous environments. Despite this rigorous legal foundation, on-site adherence remains a profound societal and industrial challenge due to worker discomfort and a lack of continuous supervision.

The domestic socioeconomic consequences of non-compliance are immense. In 2025, Vietnam recorded 7,004 occupational accidents, resulting in 658 fatalities and 1,779 severe injuries. According to the Ministry of Home Affairs, the total economic cost associated with these incidents exceeded 14,068 billion VND, encompassing medical expenses, compensation, and severe disruptions to enterprise production, alongside direct property damage exceeding 25 billion VND [3]. Solving this pressing domestic challenge requires a transition from reactive legal enforcement to proactive, automated safety monitoring.

1.2 Problem Statement

To resolve the limitations of cloud computing, the paradigm must shift toward Edge Computing (Edge AI). By deploying deep learning models directly on localized, resource-constrained edge devices (such as Raspberry Pi or NVIDIA Jetson), the visual data is processed entirely on-site. However, deploying complex Deep Learning models on embedded platforms introduces severe computer engineering bottlenecks. Edge devices are strictly constrained by low computational power, restricted thermal envelopes, and narrow memory bandwidth.

Consequently, raw Deep Learning models (operating in 32-bit floating-point precision) suffer from severe frame-dropping and unacceptable latency when executed directly on edge hardware. To achieve a viable real-time smart surveillance system for construction sites, this project must

solve two explicit technical problems:

1. **Algorithmic Optimization and Hardware Acceleration:** Standard object detection models are too computationally expensive. The challenge is to select a lightweight architecture and apply aggressive post-training optimizations (such as FP16/INT8 quantization) to maximize inference throughput ($\geq$ 20 FPS) without sacrificing the precision required to reliably identify PPE objects at a distance.

2. **Asynchronous System Integration and IoT Telemetry:** Executing video decoding, neural inference, and network transmission sequentially on a single thread inevitably causes buffer overflow. The engineering challenge is to design a concurrent, multi-threaded pipeline where physical camera ingestion, local AI processing, and lightweight MQTT telemetry operate asynchronously.

1.3 Proposed System Architecture and Objectives

This project proposes an end-to-end **IoT and Edge AI-based Smart Surveillance System** specifically designed for construction sites. The decentralized architecture operates across three primary layers:

1. **Data Acquisition Layer:** Utilizes physical cameras to capture continuous visual data of workers entering active construction zones.

2. **Edge Processing Layer:** An embedded device (Raspberry Pi/NVIDIA Jetson) executes an optimized deep learning inference engine to detect whether workers are fully equipped with **hard hats and safety vests**.

3. **IoT Communication Layer:** The edge device acts as an MQTT Publisher, broadcasting lightweight JSON payloads (containing compliance alerts) to a centralized dashboard. If a worker lacks any required PPE, the system triggers localized actuators (e.g., sirens or warning lights) via the edge device's GPIO pins.

The project objectives for the two-semester timeline include: surveying Edge AI platforms; curating a specialized dataset for hard hats and vests; training and quantizing the deep learning model; developing the multi-threaded MQTT pipeline; deploying the system on embedded hardware; and conducting comprehensive quantitative benchmarking (mAP, FPS, latency, and resource utilization).

2 Theoretical Foundations

2.1 Theoretical Basis of Object Detection Architectures

Deep learning-based object detection algorithms are fundamentally categorized into two paradigms: Two-Stage Detectors and One-Stage Detectors.

2.1.1 Two-Stage vs. One-Stage Detectors

Two-stage networks (e.g., Faster R-CNN or Mask R-CNN) separate the detection process into two distinct phases. First, a Region Proposal Network (RPN) extracts thousands of potential bounding box proposals. Second, these proposals are fed into a classification network. While yielding high accuracy, the computational complexity of evaluating thousands of regions makes them infeasible for real-time execution on resource-constrained edge devices.

Conversely, One-Stage Detectors (e.g., YOLO and SSD) formulate bounding box regression and class probability prediction as a single, unified regression problem. The input image is divided into a spatial grid, and the network predicts bounding boxes and confidence scores for each grid cell simultaneously in a single forward pass. This dense prediction architecture dramatically reduces computational overhead, achieving the high throughput (FPS) demanded by real-time video surveillance.

2.1.2 Detection Loss and Optimization Metrics

Training a one-stage detector involves minimizing a composite loss function that simultaneously penalizes classification errors and bounding box localization errors. Instead of complex mathematical derivations, the core theoretical principle relies on optimizing two metrics:

- **Classification Confidence:** Utilizing cross-entropy mechanisms to ensure the network accurately distinguishes between compliant and non-compliant objects (e.g., distinguishing a hard hat from a regular baseball cap, or a safety vest from a standard jacket).
- **Bounding Box Regression:** Modern architectures utilize variations of the Intersection over Union (IoU) metric, such as Complete IoU (CIoU). The network is penalized not only when the predicted box fails to overlap with the ground truth, but also when the center points are misaligned or the aspect ratios are inconsistent. This is crucial for tightly framing specific PPE items.

2.2 Theory of Edge Hardware and Model Quantization

Deploying deep neural networks on embedded platforms introduces severe physical constraints regarding computational power, thermal envelopes, and memory bandwidth that do

not exist in cloud environments.

2.2.1 CPU vs. GPU Embedded Architectures

- **CPU-based Edge Devices (e.g., Raspberry Pi):** Rely on ARM Cortex architectures. Inference is executed sequentially or via limited vector instructions. Lacking massive parallel processing capabilities, raw model execution is inherently slow, necessitating aggressive model compression.
- **GPU-accelerated Edge Devices (e.g., NVIDIA Jetson):** Integrate specialized hardware cores (CUDA and Tensor Cores) designed to perform highly parallelized General Matrix Multiplications (GEMM), which are the mathematical foundation of Convolutional Neural Networks (CNNs).

2.2.2 Post-Training Quantization (PTQ) Theory

Standard deep learning models are trained using 32-bit single-precision floating-point numbers (FP32). Transmitting and calculating millions of FP32 weights saturates the narrow memory bandwidth of edge devices and causes rapid thermal throttling.

Quantization is the process of mathematically mapping these continuous floating-point values to lower-precision discrete integers, typically 8-bit integers (INT8). Conceptually, this is achieved by defining a scale factor and a zero-point to linearly map the wide range of FP32 values into a compact $[-128, 127]$ integer range.

By quantizing the model to INT8, the physical memory footprint is reduced by 75%, and the hardware can utilize highly efficient integer arithmetic logic units (ALUs). This massively increases inference throughput (FPS) while significantly reducing power consumption and heat generation.

2.3 Internet of Things (IoT) Telemetry: The MQTT Protocol

To integrate the Edge AI node with a centralized dashboard without introducing severe latency, the system requires a lightweight communication protocol. Traditional HTTP (Hypertext Transfer Protocol) relies on a synchronous Request/Response model, carrying heavy header overheads that consume bandwidth and processing cycles.

MQTT (Message Queuing Telemetry Transport) is an asynchronous, publish/subscribe messaging transport protocol specifically designed for resource-constrained IoT devices and low-bandwidth networks.

- **Publish/Subscribe Architecture:** The Edge AI node acts as a *Publisher*, sending JSON payloads containing compliance alerts to specific topics (e.g., `site/zone1/alerts`).

A centralized server acts as a *Broker* to route these messages. The monitoring dashboard acts as a *Subscriber*, receiving only the data it requests.

- **Quality of Service (QoS):** MQTT guarantees delivery through adjustable QoS levels. For critical safety alerts, QoS 1 (At least once) or QoS 2 (Exactly once) ensures that a PPE violation is never lost due to an unstable construction site cellular network.

2.4 Theoretical Requirements for Computer Vision Datasets

The operational reliability of any deep learning model is strictly bounded by the statistical distribution of its training data. In the context of industrial PPE surveillance, curating a robust dataset is a highly complex engineering task. The dataset must theoretically account for several visual challenges specific to construction sites:

1. **Steep Oblique Viewing Angles:** Surveillance cameras are conventionally mounted at high elevations, creating severe downward viewing angles. The dataset must feature diverse perspectives to prevent the model from failing when viewing workers' heads and shoulders from above.

2. **Small Object Resolution:** In wide-angle views, a human occupies a small pixel area, and a hard hat occupies a minimal footprint (often $< 32 \times 32$ pixels). The network must be trained on high-density spatial data to extract fine-grained features without generating false positives against background machinery.

3. **Severe Occlusion Dynamics:** Construction environments feature scaffolding, equipment, and dynamic materials that frequently obscure portions of workers' bodies. The dataset must incorporate substantial intra-class occlusions to teach the model robust partial-feature representations.

4. **Class Imbalance and Custom Annotation:** While open-source datasets provide ample examples of hard hats, safety vests are notoriously underrepresented or inconsistently labeled. The theoretical challenge involves merging diverse datasets, rectifying label formatting disparities (e.g., COCO JSON vs. YOLO TXT), and synthesizing data augmentation techniques (like Mosaic scaling and HSV jittering) to balance the class distribution before training begins.

3 Theoretical Foundations of Deep Learning and System Architecture

3.1 Evolution of Object Detection Paradigms

Before the advent of single-stage detectors, object detection relied heavily on two-stage architectures.

- **R-CNN (Region-based Convolutional Neural Networks):** This pioneering approach extracts thousands of region proposals from an image using selective search algorithms. Each proposed region is independently warped and fed into a Convolutional Neural Network (CNN) to extract features, followed by a classifier. This decoupled pipeline is computationally expensive and prohibitively slow for real-time applications.
- **Fast R-CNN:** An optimization over R-CNN, this architecture processes the entire image through the CNN first to generate a global feature map. Region of Interest (RoI) pooling is then used to extract fixed-size feature vectors for each region proposal directly from this global map. While significantly faster, it still relies on a separate region proposal mechanism, maintaining a computational bottleneck.

3.2 Core Components of Modern Object Detection

The shift to real-time detection required a unified approach, replacing complex multi-stage pipelines with streamlined neural network architectures. The fundamental primitives of these modern systems include:

- **Grid Division:** The input image is divided into an $S \times S$ spatial grid. The core operational rule is that if the geometric center of an object falls into a specific grid cell, that cell assumes sole responsibility for detecting the object.
- **Bounding Box and Confidence Score:** A bounding box defines the spatial coordinates (center, width, and height) of a detected object. The confidence score represents the model's statistical certainty that a box contains an object, calculated mathematically as the probability of an object existing multiplied by the Intersection over Union (IoU) between the predicted box and the ground truth.
- **Anchor Boxes:** Pre-defined, statistically clustered bounding box templates (often generated via K-Means clustering on the training dataset) representing the most common object aspect ratios and sizes. Instead of predicting a box from scratch, the network predicts dimensional offsets to fit these anchors to the target object, effectively resolving issues where multiple objects overlap at the same grid center.

- **Anchor-based vs. Anchor-free:** Anchor-based models rely on predefined templates to guide bounding box regression, which improves accuracy but increases computational overhead due to the massive number of anchors evaluated. Anchor-free models eliminate these templates, directly predicting the distances from a center point to the four edges of the object. Modern mathematical assignments have made anchor-free models the standard for maximizing inference speed.
- **NMS (Non-Maximum Suppression):** A post-processing algorithm that eliminates redundant bounding boxes. It filters out low-confidence predictions and merges overlapping boxes predicting the same object, retaining only the box with the highest confidence score.
- **DFL (Distribution Focal Loss):** A loss function utilized in recent architectures to model the continuous distribution of bounding box locations. While it improves fine-grained localization, it significantly increases computational load. Eliminating DFL simplifies the detection head and reduces complexity while maintaining an unconstrained regression range[7, 8].
- **Advanced Optimization (MuSGD & STAL):** Modern training pipelines utilize hybrid optimizers like MuSGD (combining SGD with Muon, inspired by large language models) to improve optimization stability[7, 8]. Furthermore, mechanisms like Scale-Targeted Attention Labeling (STAL) are deployed to maintain positive label coverage specifically for extremely small objects[7, 8].

3.3 Anatomy of a Neural Network Architecture

State-of-the-art detectors are modularized into three distinct sequential components:

- **Backbone (Feature Extractor):** The primary convolutional network responsible for ingesting the raw image and extracting hierarchical features. It progressively identifies basic patterns (edges, colors) at shallow layers and complex semantic structures (object parts) at deeper layers.
- **Neck (Feature Aggregator):** A transitional structure, typically utilizing Feature Pyramid Networks (FPN) and Path Aggregation Networks (PANet), designed to fuse features across different scales. It combines high-level semantic understanding from top layers with fine-grained spatial details from bottom layers, enabling the network to accurately detect both massive and minuscule objects.
- **Head (Predictor):** The final decision-making component of the network. It processes the enriched feature maps from the Neck and outputs the final bounding box coordinates (Regression) and class probabilities (Classification). Advanced architectures utilize a "Decoupled Head," separating the mathematical optimization paths for regression and classification to prevent conflicting gradient updates.

3.4 Architectural Evolution of YOLO Models

3.4.1 YOLOv1 (2016): The "You Only Look Once" Paradigm

Operational Mechanics:
Introduced in 2016, the original YOLO architecture revolutionized the computer vision landscape by reframing object detection from a disjointed, multi-stage classification pipeline into a single, unified regression problem [4]. Instead of extracting region proposals, the system processes the entire image through a single Convolutional Neural Network (CNN) in one evaluation pass—literally "looking once."

The core operational flow is defined by the following steps:

- **Grid Division:** The input image is systematically divided into an $S \times S$ spatial grid (e.g., 7×7, yielding 49 distinct cells).
- **Responsibility Assignment:** The network strictly adheres to a "center-point" rule: if the geometric center of an object falls into a specific grid cell, that cell is exclusively responsible for detecting the object.
- **Unified Prediction:** Every single grid cell simultaneously predicts B bounding boxes (generating x, y center coordinates and w, h dimensions directly from scratch), confidence scores for those boxes, and C conditional class probabilities.
- **Post-Processing:** The raw output tensor contains numerous overlapping boxes. The system applies Non-Maximum Suppression (NMS) to eliminate low-confidence predictions and merge redundant boxes, yielding the final localized objects.

Key Advantages (Improvements over Prior Models):

- **Unprecedented Real-Time Speed:** By eliminating the computational bottleneck of region proposal algorithms (like selective search in R-CNN), YOLOv1 achieved state-of-the-art speeds. The base model processed images at 45 frames per second (FPS), while a smaller "Fast YOLO" variant reached 155 FPS, making true real-time video inference possible.
- **Global Contextual Understanding:** Unlike sliding window approaches or regional extractors that analyze isolated image patches, YOLOv1 processes the image globally. This macroscopic perspective enables the network to implicitly encode contextual information, effectively halving the number of false positive background errors compared to two-stage models like Fast R-CNN.

Architectural Limitations:

- **Severe Spatial Constraints:** The most critical flaw of YOLOv1 stems from its rigid grid rules. Each grid cell is mathematically restricted to predicting only a single class label. Consequently, if multiple small objects (e.g., a flock of birds or a dense crowd of people) stand too close together and share the same grid cell center, the network is forced to ignore all but one object.
- **Localization Inaccuracies:** Because the network learns to predict bounding box dimensions entirely from scratch without geometric priors, it struggles to generalize to objects with novel or unusual aspect ratios, resulting in lower spatial localization accuracy.

3.4.2 YOLOv2 (2017): The Anchor Box Revolution

Operational Mechanics:
Introduced to address the severe localization and spatial limitations of its predecessor, YOLOv2 (also known as YOLO9000) marked a fundamental paradigm shift from an anchor-free to an anchor-based architecture [5]. The rigid fully connected layers at the end of the YOLOv1 network were removed, transforming the model into a Fully Convolutional Network (FCN).

The operational advancements are characterized by the following mechanisms:

- **K-Means Clustering for Anchor Priors:** Instead of manually selecting anchor box dimensions, YOLOv2 runs a K-Means clustering algorithm directly on the training dataset's bounding boxes. This statistical approach automatically generates the top K (typically 5) most common aspect ratios and sizes, providing the network with highly optimized, data-driven templates.
- **Anchor-Based Prediction:** Every grid cell is now equipped with these K predefined anchor boxes. Rather than predicting arbitrary shapes from scratch, the network predicts dimensional offsets (shifts in center coordinates, width, and height) relative to these anchor templates.
- **Independent Classification:** The restrictive "one class per grid cell" rule was abolished. In YOLOv2, each of the K anchor boxes within a single grid cell independently predicts its own objectness confidence and class probabilities.

Key Advantages (Improvements over Prior Models):

- **Resolution of Spatial Constraints:** By delegating prediction authority to multiple anchor boxes per cell, YOLOv2 successfully detects objects that overlap or stand in dense clusters (e.g., adjacent pedestrians or grouped vehicles). If two objects share the same geometric center but have different shapes, they are seamlessly captured by different anchor templates.

- **Enhanced Localization and Stability:** Predicting parameterized offsets from predefined anchors is mathematically simpler for the neural network to optimize than predicting absolute pixel coordinates, leading to a significant increase in spatial accuracy and Mean Average Precision (mAP).
- **Batch Normalization:** The integration of batch normalization across all convolutional layers stabilized the training process, improved convergence rates, and provided a regularization effect that eliminated the need for dropout layers.

Architectural Limitations:

- **Hyperparameter Complexity:** The introduction of anchor boxes brought new complexities. Engineers had to determine the optimal number of anchors (K) and rely on dataset-specific clustering, making the model less adaptable out-of-the-box to entirely new datasets with vastly different object shapes.
- **Computational Overhead:** Evaluating thousands of anchor boxes across the entire feature map increased the computational burden, sparking future research into more efficient, anchor-free assignment mechanisms.

3.4.3 YOLOv8 (2023): Modular Architecture and the Return to Anchor-Free

Operational Mechanics:
Developed by Ultralytics, YOLOv8 represents a sophisticated maturation of the YOLO lineage [6]. It explicitly abandons the anchor-based approach of its immediate predecessors, reverting to a highly optimized, anchor-free paradigm. The architecture is systematically modularized into three distinct operational factories:

- **Advanced Backbone (Feature Extraction):** The network utilizes a modified CSP-Darknet backbone featuring the C2f module. This design splits the gradient flow, allowing the model to extract richer semantic features from the input image while minimizing redundant computational overhead. At the terminus of the backbone, a Spatial Pyramid Pooling - Fast (SPPF) module expands the receptive field, enabling the network to perceive global context efficiently.
- **Neck (Multi-Scale Aggregation):** YOLOv8 employs a combined Feature Pyramid Network (FPN) and Path Aggregation Network (PANet) architecture. The FPN passes high-level semantic understanding top-down, while the PANet propagates low-level spatial and localization details bottom-up. This bidirectional fusion generates distinct feature maps at different scales, ensuring the robust detection of both massive and minuscule objects.

- **Decoupled Head:** Unlike older versions (such as YOLOv1 and YOLOv2) that utilized a single shared head for all predictions, YOLOv8 physically separates the mathematical optimization paths. One branch is strictly dedicated to object classification (identifying the class), while a parallel, independent branch handles bounding box regression (locating the coordinates).
- **Task-Aligned Assigner (TAL):** Replacing static anchor boxes, TAL dynamically evaluates predictions during the training phase based on a combined metric of classification accuracy and bounding box alignment. It intelligently selects and rewards the network's most accurate spatial and semantic predictions without relying on predefined templates.

Key Advantages (Improvements over Prior Models):

- **Anchor-Free Elegance:** By eliminating anchor boxes, YOLOv8 drastically reduces the number of heuristic hyperparameters engineers need to manually tune. The network becomes inherently more flexible and capable of generalizing to irregularly shaped objects.
- **Resolved Task Conflict:** The Decoupled Head resolves the inherent mathematical conflict between classification (which requires translation invariance to recognize an object regardless of its position) and regression (which requires translation variance to pinpoint exact coordinates). This separation leads to significantly faster convergence and higher Mean Average Precision (mAP).

Architectural Limitations:

- **Post-Processing Bottleneck (NMS):** Despite its advanced internal neural design, YOLOv8 still relies heavily on the Non-Maximum Suppression (NMS) algorithm to filter out redundant bounding boxes. This post-processing step creates a noticeable latency bottleneck, particularly when the model is deployed on resource-constrained Edge devices without powerful GPUs.
- **DFL Computational Overhead:** The utilization of Distribution Focal Loss (DFL) for precise bounding box regression requires the network to compute continuous probability distributions. While this improves spatial accuracy, it adds floating-point operations (FLOPs) that hinder raw inference speed on standard CPUs.

3.4.4 YOLO26 (2026): Unified End-to-End Vision Models

Operational Mechanics:
Released in January 2026, Ultralytics YOLO26 represents a massive architectural leap by unifying real-time performance with a truly end-to-end processing pipeline[7, 8]. It retains the anchor-free paradigm but fundamentally overhauls the head design and training recipe:

- **Dual-Head Architecture:** To provide deployment flexibility, YOLO26 utilizes a dual-head design[7, 8]. It features a default "one-to-many" head that utilizes NMS for maximum accuracy, alongside an optional "one-to-one" head that generates predictions natively end-to-end without requiring NMS (configured via `nms=False`)[7, 8].
- **DFL-Free Regression:** The architecture explicitly eliminates Distribution Focal Loss (DFL)[7, 8]. This design choice significantly reduces the complexity of the detection head while successfully maintaining an unconstrained regression range[7, 8].
- **Advanced Training Recipe:** The training pipeline introduces MuSGD, a hybrid optimizer combining standard SGD with Muon—a technique adapted from Large Language Models (LLMs)[7, 8]. Furthermore, it integrates Progressive Loss to shift the optimization focus towards inference-time structures, and Scale-Targeted Attention Labeling (STAL) to maintain dense positive label coverage for extremely small objects[7, 8].
- **Open-Vocabulary Expansion:** The lineage introduces YOLOE-26, an open-vocabulary variant capable of detecting and segmenting novel object categories via text prompts, visual prompts, or prompt-free modes, without being restricted to fixed training classes[7, 8].

Key Advantages (Improvements over Prior Models):

- **Eradication of the Post-Processing Bottleneck:** By supporting native end-to-end inference without NMS, YOLO26 dramatically simplifies deployment pipelines[7, 8]. This streamlined architecture, combined with the DFL-free regression, enables YOLO26n to achieve up to a 43% faster ONNX inference speed on standard CPUs compared to its predecessor, YOLO11n[7, 8].
- **Unified Multi-Task Supremacy:** A single, cohesive pipeline now supports advanced task-specific heads, yielding substantial improvements such as +7.2 AP in pose estimation (via Residual Log-Likelihood Estimation) and +3.4 mAP in oriented bounding box (OBB) detection[7, 8].

Architectural Limitations:

- **Accuracy vs. Latency Trade-off in E2E:** While the NMS-free (one-to-one) head maximizes speed and simplifies edge deployment, the traditional one-to-many head (with NMS) still yields slightly higher absolute Mean Average Precision (mAP) metrics, requiring engineers to make a conscious trade-off based on hardware constraints[7, 8].

3.5 Comprehensive Architectural Comparison

Version	Paradigm	Core Architecture / Technologies	Major Breakthroughs	Key Limitations
YOLOv1 (2016)	Anchor-free	Unified CNN, $S \times S$ spatial grid division, fully connected terminal layers [4].	Real-time inference speed; global contextual reasoning reduces background errors.	Severe spatial constraints; fails to detect overlapping objects or dense clusters.
YOLOv2 (2017)	Anchor-based	Fully Convolutional Network (FCN), K-Means clustered Anchor Boxes, Batch Normalization [5].	Resolved spatial conflicts; enables multi-object detection within a single grid cell.	Computational overhead from evaluating thousands of anchors; requires manual hyperparameter tuning.
YOLOv8 (2023)	Anchor-free	Backbone-Neck-Head factory design, Decoupled Head, Task-Aligned Assigner (TAL) [6].	Eliminates anchor boxes entirely; Decoupled Head resolves classification/regression conflicts.	Heavy reliance on NMS post-processing; DFL module adds computational latency on CPUs.
YOLO26 (2026)	Anchor-free	Dual-Head design, NMS-free native end-to-end inference, MuSGD optimizer, YOLOE-26 integration[7, 8].	43% CPU speedup via DFL removal[7, 8]; pure end-to-end deployment without NMS bottleneck[7, 8].	Minor accuracy drop when operating in strictly NMS-free mode compared to traditional mode[7, 8].

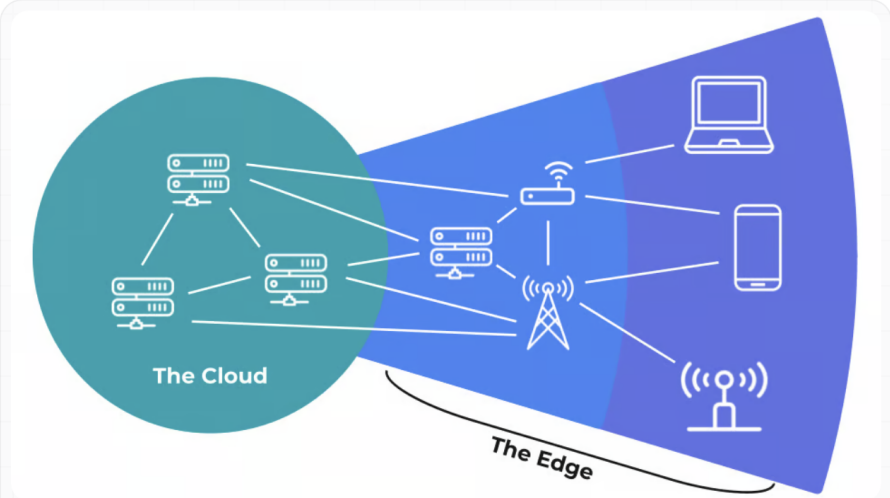
Table 1: Comprehensive technological evolution of YOLO architectures from v1 to YOLO26.

4 Edge Platforms, Model Optimization

4.1 Edge computing

What is edge computing?
Edge computing is a strategy where data is processed and stored at the periphery of a computer network. In most cases, processing and storing data on remote servers, especially internet servers, is known as "cloud computing." The edge includes all computing devices not part of the cloud.

In general, data will be created by end-point devices. "End-point devices" or "end devices" refer to physical equipment at the very edge of the network, such as laptops, smartphones, and connected sensors. Sometimes, these end devices have a user interface where a person can interact with various applications, enter data, etc. Other times, the device is embedded into other equipment or offers no user interface. These embedded devices, if connected to the internet or other networks, are referred to as the Internet of Things.



Cloud computing vs. edge computing

What is edge AI?
Edge AI is the deployment of AI applications in devices throughout the physical world. It's called "edge AI" because the AI computation is done near the user at the edge of the network, close to where the data is located, rather than centrally in a cloud computing facility or private data center.

The traditional approach to AI and machine learning (ML) is to use powerful, cloud-based servers to perform model training as well as inference (prediction serving). While edge devices

might have limited resources compared to their cloud-based cousins, they offer reduced bandwidth usage, lower latency, and additional data privacy.

4.2 Edge devices

Raspberry Pi
Raspberry Pi is the name of a series of single-board computers made by the Raspberry Pi Foundation, a UK charity that aims to educate people in computing and create easier access to computing education.

While it's often used for DIY projects, home automation, and learning to code, the Raspberry Pi also serves as a versatile platform for AI projects. With multiple models available, the Raspberry Pi brings accessible computing power to anyone looking to experiment, create, and innovate without breaking the bank.

Nvidia Jetson
NVIDIA Jetson is the platform for robotics and edge AI applications, offering compact edge AI computers, supported by the NVIDIA JetPack SDK for accelerated software development.

The Jetson Nano is a powerful, compact computing platform developed by NVIDIA. It is designed specifically for AI and machine learning applications. It's part of the Jetson family, which provides developers with the ability to build AI-powered systems and robots. Despite its small size, the Nano packs a punch with a quad-core ARM processor and a 128-core GPU, making it ideal for tasks like image recognition, object detection, and robotics.

Comparison
In conclusion, both the Jetson Nano and Raspberry Pi offer unique advantages depending on the specific requirements of your AI projects. While the Jetson Nano excels in handling demanding AI tasks with its powerful GPU and optimized software, the Raspberry Pi remains a versatile and cost-effective choice for simpler AI and DIY projects.

Category	NVIDIA Jetson Nano	Raspberry Pi
AI Performance	Delivers significantly superior AI processing power for complex tasks.	Excels in ease of use, affordability, and accessibility for simpler tasks.
Target Applications	Real-time object detection/tracking Multi-stream video analytics Autonomous robotics navigation Industrial quality control systems Advanced computer vision	Basic object detection projects Educational AI demonstrations Simple computer vision tasks IoT sensor data processing Home automation with AI features
Software Ecosystem	Comprehensive support via JetPack SDK (CUDA, cuDNN, TensorRT). Deep learning frameworks leverage GPU acceleration.	Broader compatibility with multiple OS and languages. Extensive educational resources, but limited AI optimizations.

4.3 Edge AI deployment

At the core of Edge AI is the inference process, where pre-trained machine learning models are deployed on edge hardware. These models are optimized for resource-constrained environments and perform tasks such as classification, anomaly detection, or predictive analysis.

The typical workflow includes:

Data acquisition from sensors or connected devices Preprocessing and filtering at the edge Local AI inference using embedded or edge-based models Selective data transmission to cloud systems for aggregation or retraining Communication between devices and systems relies on IoT protocols such as MQTT, CoAP, or HTTP, depending on the application requirements. Edge AI systems often integrate with cloud platforms for model updates, orchestration, and long-term analytics.

Key technologies and standards

Edge hardware: AI-enabled microcontrollers (MCUs), system-on-chip (SoC) platforms, and

- **Edge hardware:** AI-enabled microcontrollers (MCUs), system-on-chip (SoC) platforms, and specialized accelerators such as GPUs, NPUs, and TPUs
- **Machine learning frameworks:** TensorFlow Lite, PyTorch Mobile, ONNX Runtime, and vendor-specific SDKs for embedded AI
- **Connectivity protocols:** MQTT, CoAP, HTTP/REST for data exchange between edge devices and cloud systems

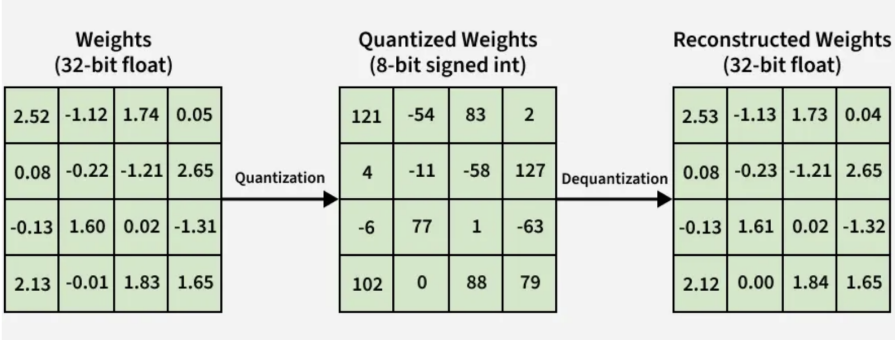
- **Edge computing platforms:** Local gateways and industrial PCs that aggregate data and host AI workloads
- **Model optimization techniques:** Quantization, pruning, and compression to reduce model size and computational requirements

4.4 Model optimization

Inference mechanisms are the computational and logical procedures by which an intelligent system derives new conclusions from existing knowledge. Within artificial intelligence and knowledge-based systems, an inference mechanism is the engine that applies rules or probabilistic models to a stored knowledge base, producing answers to queries or triggering actions.

4.4.1 Quantization

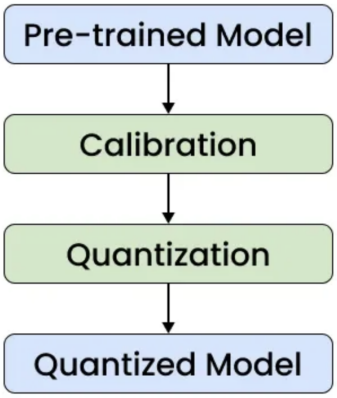
Quantization is a model optimization technique that reduces the precision of numerical values such as weights and activations in models to make them faster and more efficient. It helps lower memory usage, model size, and computational cost while maintaining almost the same level of accuracy.



Types of Quantization
Post-Training Quantization (PTQ)

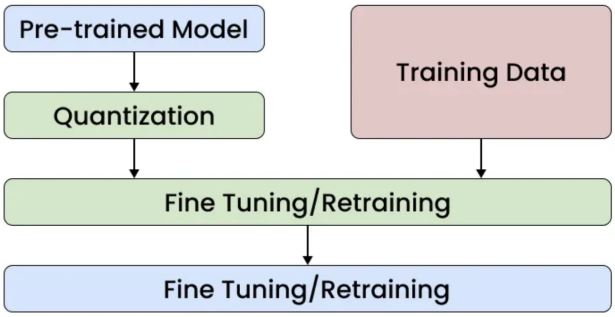
- Quantization is applied after the model is fully trained, converting high-precision parameters (like FP32 weights and activations) into lower-bit formats such as INT8 to reduce computational load and memory usage.
- The two primary techniques are Static Quantization which is pre-calibrated with representative data and Dynamic Quantization where activations are quantized on the fly during inference.

- Enables faster inference and smaller model size without retraining, making it ideal for deploying large language models on edge devices or latency-sensitive environments.



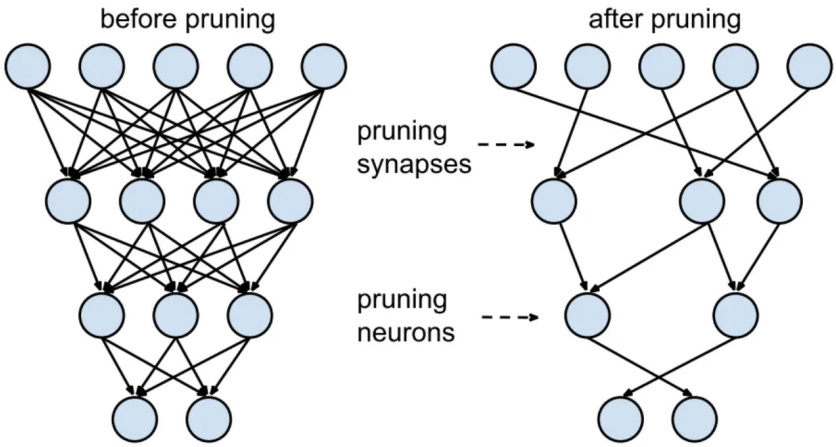
Quantization-Aware Training (QAT)

- The model is trained with quantization in mind, simulating low-precision operations during training.
- Helps the model adapt to quantization effects and minimize accuracy loss.
- Delivers higher accuracy than Post-Training Quantization (PTQ) by learning to handle reduced precision.



4.4.2 Pruning

Pruning is a process of removing redundant or less important neurons or connections (weights) from a neural network without significantly impacting its performance. By eliminating these unnecessary components, the model's size and complexity can be reduced, leading to faster inference times and lower memory usage.



Types of Pruning
Weight Pruning (Unstructured Pruning)

- Magnitude-based Pruning: In this method, weights with small magnitudes (close to zero) are considered less important and are pruned. The assumption is that these weights have minimal impact on the model's predictions.
- Random Pruning: Here, weights are pruned randomly without considering their value. This technique is not commonly used due to the risk of removing important weights.

Structured Pruning

- Filter Pruning: This method prunes entire convolutional filters in CNNs. Filters that are not contributing significantly to the output are removed, leading to smaller and faster models.
- Neuron Pruning: Neurons that are contributing less to the output are pruned. This is particularly useful in fully connected layers, where many neurons may be redundant.
- Layer Pruning: In some cases, entire layers or blocks of layers can be pruned, especially if they are contributing little to the model's accuracy. However, this technique is more aggressive and can result in a significant loss of performance if not done carefully.

Dynamic Pruning
In dynamic pruning, the pruning process is adaptive and occurs during training rather than after the model has been fully trained. This allows the model to learn which connections are important and which can be removed as training progresses.

References

[1] P. Sanjeewani, G. Neuber, J. Fitzgerald, N. Chandrasena, S. Potums, A. Alavi, and C. Lane, "Real-Time Personal Protective Equipment Non-Compliance Recognition on AI Edge Cameras," *Electronics*, vol. 13, p. 2990, Jul. 2024, doi: 10.3390/electronics13152990.

[2] National Assembly of Vietnam, "Law on Occupational Safety and Health," Law No. 84/2015/QH13, Hanoi, Vietnam, 2015.

[3] Ministry of Home Affairs, "Tình hình tai nạn lao động năm 2025: Giảm về số vụ nhưng vẫn tiềm ẩn nhiều rủi ro," *Journal of State Organization and Labour*, Apr. 03, 2026.

[4] Redmon, Joseph and Divvala, Santosh and Girshick, Ross and Farhadi, Ali, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision a" Pattern Recognition (CVPR)*, pp. 779-788, 2016.

[5] GeeksforGeeks, "YOLO v2 Object Detection," [Online]. Available: https://www.geeksforgeeks.org/machine-learning/yolo-v2-object-detection/, 2024.

[6] Yaseen, Muhammad, "What is YOLOv8: An in-depth exploration of the internal features of the next-generation object detector," *arXiv preprint arXiv:2408.15857*, Aug. 2024.

[7] Ultralytics, "YOLO26 Ultralytics Docs," [Online]. Available: https://docs.ultralytics.com/vi/models/yolo26, 2026.

[8] Hidayatullah, Priyanto and Tubagus, Refdinal, "YOLO26: A comprehensive architecture overview and key improvements," *arXiv preprint arXiv:2602.14582*, Feb. 2026.

[9] Edge Impulse. (2024, May 1). *What is Edge AI?* [Video]. YouTube. https://youtu.be/QXuvUW4B7jU?si=SS6x5w3CWs4lEv13

[10] NVIDIA. (n.d.). *What is edge AI?* NVIDIA Blog. https://blogs.nvidia.com/blog/what-is-edge-ai/

[11] Edge Impulse. (n.d.). *Intro to edge AI.* Edge Impulse Documentation. https://docs.edgeimpulse.com/knowledge/courses/edge-ai-fundamentals/intro-to-edge-ai

[12] IoT Business News. (2026, April 1). *Edge AI for IoT: Use cases, benefits and deployment challenges.* IoT Business News. https://iotbusinessnews.com/2026/04/01/edge-ai-for-iot-use-cases-benefits-and-deployment-challenges/

[13] Think Robotics. (n.d.). *Jetson Nano vs Raspberry Pi AI: The ultimate performance comparison for edge computing.* Think Robotics. https://thinkrobotics.com/blogs/learn/jetson-nano-vs-raspberry-pi-ai-the-ultimate-performance-comparison-for-edge-computi

[14] Two Win Tech. (n.d.). *Jetson Nano vs. Raspberry Pi: Which is better for AI projects?* Two Win Tech. https://twowintech.com/jetson-nano-vs-raspberry-pi-which-is-better-for-ai-projects/

[15] GeeksforGeeks. (n.d.). *Quantization in deep learning.* GeeksforGeeks. https://www.geeksforgeeks.org/deep-learning/quantization-in-deep-learning/

[16] GeeksforGeeks. (n.d.). *Neural network pruning in deep learning.* GeeksforGeeks. https://www.geeksforgeeks.org/deep-learning/neural-network-pruning-in-deep-learning/

[17] Paul, S. (n.d.). *Pruning in deep learning models.* Medium. https://medium.com/@souvik.paul01/pruning-in-deep-learning-models-1067a19acd89